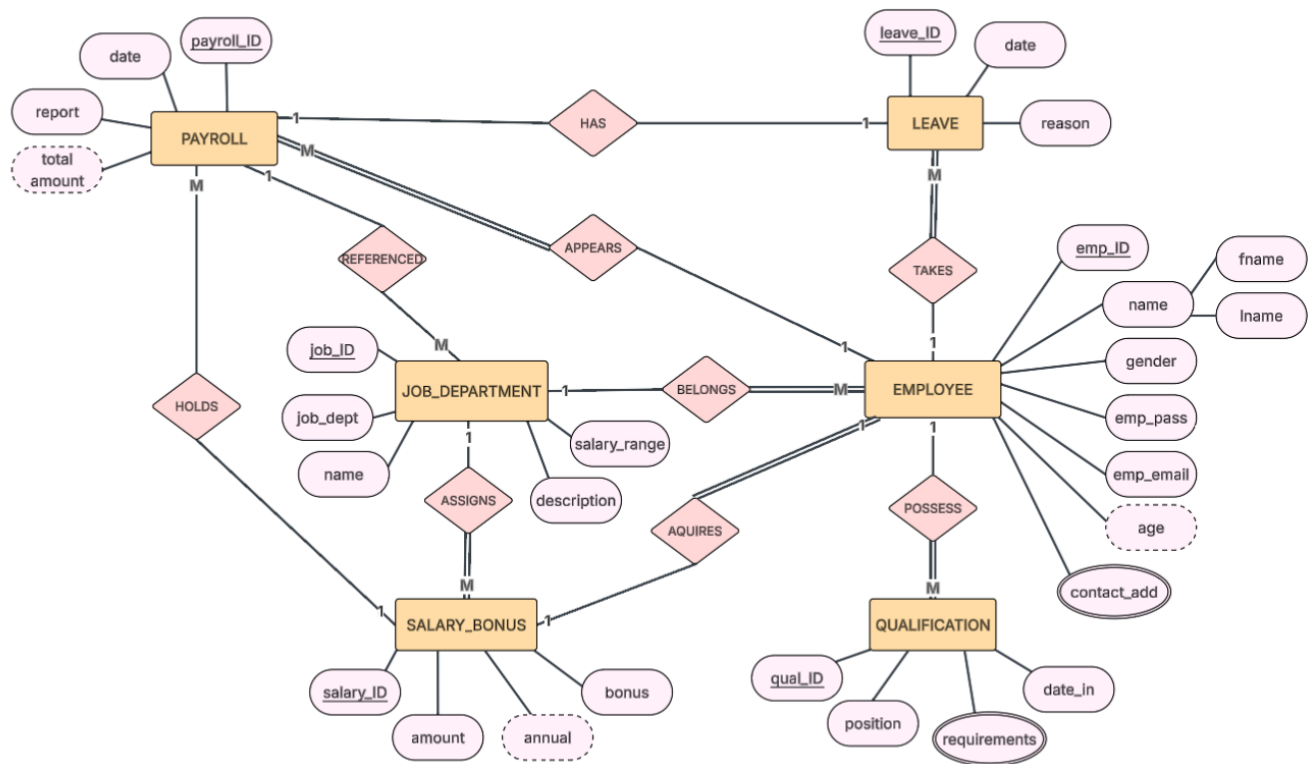


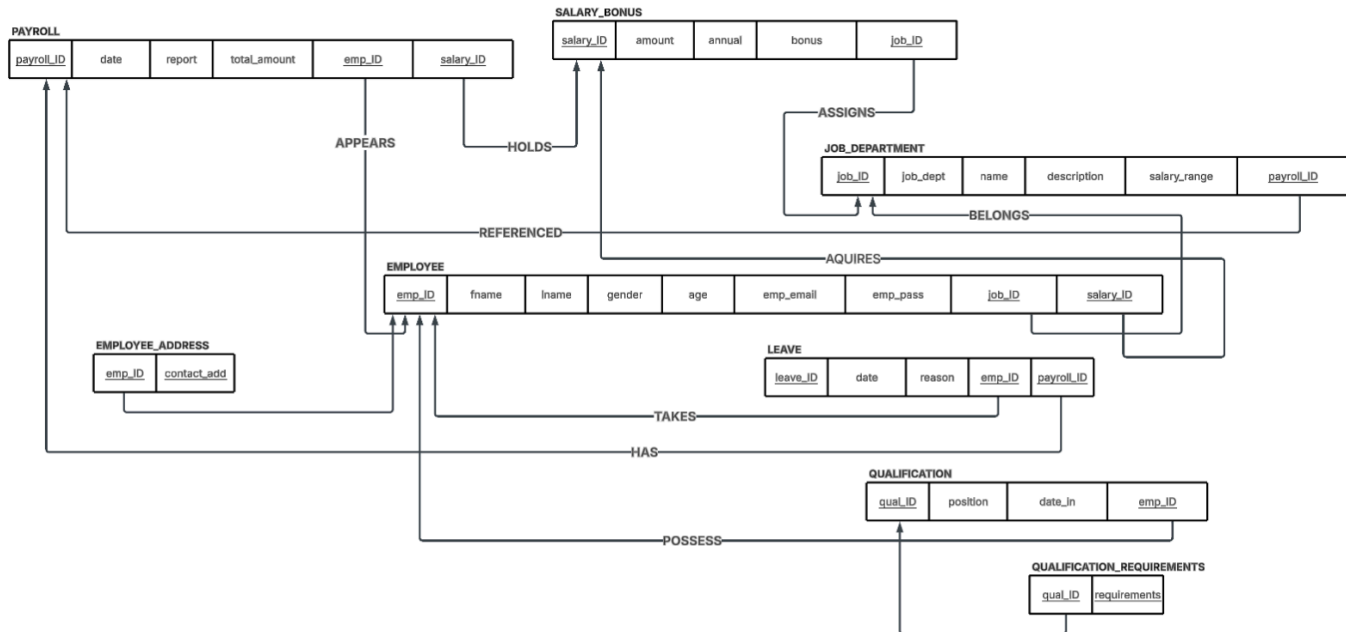
Employee Management System

01 ERD Design & Mapping

Task 1:



Task 2:



Mapping:

PAYROLL (PK_payroll_ID, date, report, total_amount, FK_emp_ID, FK_salary_ID)

SALARY_BONUS (PK_salary_ID, amount, annual, bonus, FK_job_ID)

JOB_DEPARTMENT (PK_job_ID, job_dept, name, description, salary_range, FK_payroll_ID)

EMPLOYEE (PK_emp_ID, fname, lname, gender, age, emp_email, emp_pass, FK_job_ID, FK_salary_ID)

LEAVE (PK_leave_ID, date, reason, FK_emp_ID, FK_payroll_ID)

QUALIFICATION (PK_qual_ID, position, date_in, FK_emp_ID)

EMPLOYEE_ADDRESS (FK_emp_ID, PK_contact_add)

QUALIFICATION_REQUIREMENTS (FK_qual_ID, PK_requirements)

Foreign Keys:

PAYROLL.emp_ID references EMPLOYEE(emp_ID)

PAYROLL.salary_ID references SALARY_BONUS(salary_ID)

SALARY_BONUS.job_ID references JOB_DEPARTMENTS(job_ID)

JOB_DEPARTMENTS.payroll_ID references PAYROLL.(payroll_ID)

EMPLOYEE.job_ID references JOB_DEPARTMENTS(job_ID)

EMPLOYEE.salary_ID references SALARY_BONUS(salary_ID)

LEAVE.emp_ID references EMPLOYEE(emp_ID)

LEAVE.payroll_ID references PAYROLL.(payroll_ID)

QUALIFICATION.emp_ID references EMPLOYEE(emp_ID)

EMPLOYEE_ADDRESS.emp_ID references EMPLOYEE(emp_ID)

QUALIFICATION_REQUIREMENTS .qual_ID references QUALIFICATION(qual_ID)

Task 3:

-- TABLE 1: JOB_DEPARTMENT

CREATE SEQUENCE job_dept_seq START WITH 1 INCREMENT BY 1;

```
CREATE TABLE JOB_DEPARTMENT (  
  job_ID      NUMBER DEFAULT job_dept_seq.NEXTVAL PRIMARY KEY,  
  job_dept    VARCHAR2(50)    NOT NULL,  
  name        VARCHAR2(100)   NOT NULL,  
  description  VARCHAR2(255),  
  salary_range VARCHAR2(50),  
  payroll_ID  NUMBER  
);
```

-- TABLE 2: SALARY_BONUS

CREATE SEQUENCE salary_bonus_seq START WITH 1 INCREMENT BY 1;

```
CREATE TABLE SALARY_BONUS (  
  salary_ID   NUMBER DEFAULT salary_bonus_seq.NEXTVAL PRIMARY KEY,  
  amount      NUMBER(10,2)  NOT NULL,  
  annual      NUMBER(10,2),  
  bonus       NUMBER(10,2),  
  job_ID      NUMBER        NOT NULL,  
  CONSTRAINT chk_amount_positive CHECK (amount > 0),  
  CONSTRAINT fk_sb_job FOREIGN KEY (job_ID)  
    REFERENCES JOB_DEPARTMENT(job_ID)  
);
```

-- TABLE 3: EMPLOYEE

CREATE SEQUENCE employee_seq START WITH 1 INCREMENT BY 1;

```
CREATE TABLE EMPLOYEE (  
  emp_ID      NUMBER DEFAULT employee_seq.NEXTVAL PRIMARY KEY,  
  fname       VARCHAR2(50)    NOT NULL,  
  lname       VARCHAR2(50)    NOT NULL,  
  gender      CHAR(1)         CHECK (gender IN ('M', 'F')),  
  age         NUMBER(3),  
  emp_email   VARCHAR2(100)   NOT NULL UNIQUE,  
  emp_pass    VARCHAR2(255)   NOT NULL,  
  job_ID      NUMBER          NOT NULL,
```

```
salary_ID    NUMBER          NOT NULL,  
CONSTRAINT fk_emp_job  FOREIGN KEY (job_ID)  
REFERENCES JOB_DEPARTMENT(job_ID),  
CONSTRAINT fk_emp_salary FOREIGN KEY (salary_ID)  
REFERENCES SALARY_BONUS(salary_ID)  
);
```

-- TABLE 4: PAYROLL

```
CREATE SEQUENCE payroll_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE PAYROLL (  
  payroll_ID    NUMBER DEFAULT payroll_seq.NEXTVAL PRIMARY KEY,  
  pay_date      DATE          NOT NULL,  
  report        VARCHAR2(255),  
  total_amount  NUMBER(10,2)   NOT NULL,  
  emp_ID        NUMBER          NOT NULL,  
  salary_ID     NUMBER          NOT NULL,  
  CONSTRAINT fk_pay_emp  FOREIGN KEY (emp_ID)  
REFERENCES EMPLOYEE(emp_ID),  
  CONSTRAINT fk_pay_salary FOREIGN KEY (salary_ID)  
REFERENCES SALARY_BONUS(salary_ID)  
);
```

-- FK PAYROLL_ID TO JOB_DEPARTMENT

```
ALTER TABLE JOB_DEPARTMENT  
ADD CONSTRAINT fk_jd_payroll FOREIGN KEY (payroll_ID)  
REFERENCES PAYROLL(payroll_ID);
```

-- TABLE 5: LEAVE

```
CREATE SEQUENCE leave_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE LEAVE (  
  leave_ID      NUMBER DEFAULT leave_seq.NEXTVAL PRIMARY KEY,  
  leave_date    DATE          NOT NULL,  
  reason        VARCHAR2(500),  
  emp_ID        NUMBER          NOT NULL,
```

```

    payroll_ID    NUMBER,
    CONSTRAINT fk_leave_emp    FOREIGN KEY (emp_ID)
        REFERENCES EMPLOYEE(emp_ID),
    CONSTRAINT fk_leave_payroll FOREIGN KEY (payroll_ID)
        REFERENCES PAYROLL(payroll_ID)
);

```

-- TABLE 6: QUALIFICATION

```

CREATE SEQUENCE qualification_seq START WITH 1 INCREMENT BY 1;

```

```

CREATE TABLE QUALIFICATION (
    qual_ID    NUMBER DEFAULT qualification_seq.NEXTVAL PRIMARY KEY,
    position   VARCHAR2(100)    NOT NULL,
    date_in    DATE             NOT NULL,
    emp_ID     NUMBER           NOT NULL,
    CONSTRAINT fk_qual_emp FOREIGN KEY (emp_ID)
        REFERENCES EMPLOYEE(emp_ID)
);

```

-- TABLE 7: EMPLOYEE_ADDRESS

```

CREATE TABLE EMPLOYEE_ADDRESS (
    emp_ID     NUMBER           NOT NULL,
    contact_add VARCHAR2(255)    NOT NULL,
    CONSTRAINT pk_emp_address PRIMARY KEY (emp_ID, contact_add),
    CONSTRAINT fk_addr_emp FOREIGN KEY (emp_ID)
        REFERENCES EMPLOYEE(emp_ID)
);

```

-- TABLE 8: QUALIFICATION_REQUIREMENTS

```

CREATE TABLE QUALIFICATION_REQUIREMENTS (
    qual_ID     NUMBER           NOT NULL,
    requirements VARCHAR2(255)    NOT NULL,
    CONSTRAINT pk_qual_req PRIMARY KEY (qual_ID, requirements),
    CONSTRAINT fk_req_qual FOREIGN KEY (qual_ID)
        REFERENCES QUALIFICATION(qual_ID)
);

```

02 Normalization

Task 1:

EMP_RAW (emp_id, full_name, phone_numbers, dept_name, dept_location, job_titles, salary)

- a) The attributes phone_numbers and job_titles are multivalued attributes, an employee could have more than one phone number or more than one job title, as well as full_name since it will contain more than one name, and the first normal form states that we should eliminate repeating groups and ensure atomic (indivisible) values in each field.

b) EMP (emp_id, fname,lname, dept_name, dept_location, salary)

EMP (1, Tibyan, Saad, HR, muscat, 650)

EMP (2, Razan, Al-Harhi, HR, muscat, 650)

EMP (3, Mariya, Al-Lamki, IT, sohar, 700)

EMP_PHONE (emp_id,phone_number)

EMP_PHONE (1,98756521)

EMP_PHONE (1,73892423)

EMP_PHONE (2,7365433)

EMP_PHONE (3,98881234)

EMP_JOBS (emp_id , job_title)

EMP_JOBS (1, Recruitment)

EMP_JOBS (2, Records Manager)

EMP_JOBS (1, trainee)

EMP_JOBS (3, Network configuration)

- c) My EMS schema does satisfy the 1NF as I have removed all the multivalued attributes and placed them in separate tables that link to the main table to ensure atomic values. Here are the examples:

EMPLOYEE (PK_emp_ID, fname,lname, gender, age, emp_email, emp_pass, FK_job_ID, FK,salary_ID)

EMPLOYEE_ADDRESS (PK_FK_emp_ID, PK_contact_add)

QUALIFICATION_REQUIREMENTS (PK_FK_qual_ID, PK_requirements)

QUALIFICATION (PK_qual_ID, position,date_in,FK_emp_ID)

Task 2:

PAYROLL_RAW (emp_ID, job_ID, emp_name, job_title, salary_range, pay_date, total_amount)

- a) Emp_name depends on emp_id
job_title depends on job_id
Salary_range depends on job_id
- b) PAYROLL_EMP (emp_ID,emp_name)
PAYROLL_JOB(job_ID, job_title, salary_range)
EMP_JOB(job_ID, emp_ID, pay_date, total_amount)

EMPLOYEE_EXT (emp_ID, dept_ID, dept_name, dept_manager, manager_phone, emp_salary)

TASK 3:

- a. The non-key attribute dept_ID determines dept_name, dept_manager, and manager_phone, and dept_manager further determines manager_phone. This means dept_name, dept_manager, and manager_phone do not depend directly on the primary key emp_ID, they depend on another non-key attribute (dept_ID) instead, which is exactly what a transitive dependency is. Third normal form states that every non-key attribute must depend only on the key, the whole key, and nothing but the key, so these attributes need to be moved out into their own table.
- b. EMPLOYEE (emp_ID, dept_ID, emp_salary)
DEPARTMENT (dept_ID, dept_name, dept_manager, manager_phone)
- c. No transitive dependency remains because EMPLOYEE only holds attributes that depend directly on emp_ID, and DEPARTMENT only holds attributes that depend directly on dept_ID. dept_manager and manager_phone are no longer hiding behind emp_ID, they sit correctly under their true determinant dept_ID.
- d. My schema does satisfy the 3NF as I have checked every table and confirmed that no non-key attribute determines another non-key attribute, since all department-related details (name, description, salary_range) live only inside JOB_DEPARTMENT, and all salary-related details (amount, bonus, annual) live only inside SALARY_BONUS, rather than being repeated or derivable through another non-key attribute inside EMPLOYEE or PAYROLL.

03-SQL DML — INSERT / UPDATE / DELETE / SELECT

```
-- =====
-- 03-SQL DML — INSERT / UPDATE / DELETE / SELECT
-- =====

-- =====
-- Task 1
-- =====

-- Sequences
CREATE SEQUENCE job_dept_seq START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE salary_bonus_seq START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE employee_seq START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE payroll_seq START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE leave_seq START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE qualification_seq START WITH 1 INCREMENT BY 1;

-- 5 departments
INSERT INTO JOB_DEPARTMENT (job_dept, name, description, salary_range)
VALUES ('HR', 'Human Resources', 'Manages employee relations and recruitment',
'3000-6000');

INSERT INTO JOB_DEPARTMENT (job_dept, name, description, salary_range)
VALUES ('IT', 'Information Technology', 'Manages systems, networks and software
development', '4000-9000');

INSERT INTO JOB_DEPARTMENT (job_dept, name, description, salary_range)
VALUES ('FIN', 'Finance', 'Handles budgeting, accounting and financial reporting', '3500-7000');

INSERT INTO JOB_DEPARTMENT (job_dept, name, description, salary_range)
VALUES ('OPS', 'Operations', 'Oversees daily business operations and logistics', '3000-6500');

INSERT INTO JOB_DEPARTMENT (job_dept, name, description, salary_range)
VALUES ('MKT', 'Marketing', 'Manages branding, campaigns and market research',
'3000-7000');

-- 5 salary records
INSERT INTO SALARY_BONUS (amount, annual, bonus, job_ID)
VALUES (4500, 54000, 1000, 1);

INSERT INTO SALARY_BONUS (amount, annual, bonus, job_ID)
VALUES (7000, 84000, 2000, 2);
```



```
INSERT INTO SALARY_BONUS (amount, annual, bonus, job_ID)
VALUES (5500, 66000, 1500, 3);
```

```
INSERT INTO SALARY_BONUS (amount, annual, bonus, job_ID)
VALUES (5000, 60000, 1200, 4);
```

```
INSERT INTO SALARY_BONUS (amount, annual, bonus, job_ID)
VALUES (5200, 62400, 1300, 5);
```

-- 10 employees

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Mohammed', 'Al-Rashidi', 'M', 35, 'mohammed.rashidi@ems.com', 'Pass@1234', 1,
1);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Fatima', 'Al-Balushi', 'F', 28, 'fatima.balushi@ems.com', 'Pass@1234', 2, 2);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Ahmed', 'Al-Harathi', 'M', 42, 'ahmed.harathi@ems.com', 'Pass@1234', 3, 3);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Maryam', 'Al-Zadjali', 'F', 31, 'maryam.zadjali@ems.com', 'Pass@1234', 4, 4);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Khalid', 'Al-Farsi', 'M', 38, 'khalid.farsi@ems.com', 'Pass@1234', 5, 5);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Noura', 'Al-Habsi', 'F', 26, 'noura.habsi@ems.com', 'Pass@1234', 1, 1);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Omar', 'Al-Maskari', 'M', 33, 'omar.maskari@ems.com', 'Pass@1234', 2, 2);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID,
salary_ID)
VALUES ('Aisha', 'Al-Lawati', 'F', 29, 'aisha.lawati@ems.com', 'Pass@1234', 3, 3);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID, salary_ID)
```

```
VALUES ('Yousuf', 'Al-Mamari', 'M', 45, 'yousuf.mamari@ems.com', 'Pass@1234', 4, 4);
```

```
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID, salary_ID)
```

```
VALUES ('Hessa', 'Al-Siyabi', 'F', 30, 'hessa.siyabi@ems.com', 'Pass@1234', 5, 5);
```

```
-- 8 payroll records
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-01-31', 'YYYY-MM-DD'), 'January payroll processed', 5500, 1, 1);
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-01-31', 'YYYY-MM-DD'), 'January payroll processed', 9000, 2, 2);
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-01-31', 'YYYY-MM-DD'), 'January payroll processed', 7000, 3, 3);
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-01-31', 'YYYY-MM-DD'), 'January payroll processed', 6200, 4, 4);
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-01-31', 'YYYY-MM-DD'), 'January payroll processed', 6500, 5, 5);
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-02-28', 'YYYY-MM-DD'), 'February payroll processed', 5500, 1, 1);
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-02-28', 'YYYY-MM-DD'), 'February payroll processed', 9000, 2, 2);
```

```
INSERT INTO PAYROLL (pay_date, report, total_amount, emp_ID, salary_ID)
```

```
VALUES (TO_DATE('2025-02-28', 'YYYY-MM-DD'), 'February payroll processed', 7000, 3, 3);
```

```
-- 5 leave records
```

```
INSERT INTO LEAVE (leave_date, reason, emp_ID, payroll_ID)
```

```
VALUES (TO_DATE('2025-01-10', 'YYYY-MM-DD'), 'Medical leave for surgery recovery', 1, 1);
```

```
INSERT INTO LEAVE (leave_date, reason, emp_ID, payroll_ID)
```

```
VALUES (TO_DATE('2025-01-15', 'YYYY-MM-DD'), 'Annual vacation leave', 2, 2);
```

```
INSERT INTO LEAVE (leave_date, reason, emp_ID, payroll_ID)
```

```
VALUES (TO_DATE('2025-02-05', 'YYYY-MM-DD'), 'Family emergency', 3, 3);
```

```
INSERT INTO LEAVE (leave_date, reason, emp_ID, payroll_ID)
```

```
VALUES (TO_DATE('2025-02-12', 'YYYY-MM-DD'), 'Maternity leave', 4, 4);
```

```
INSERT INTO LEAVE (leave_date, reason, emp_ID, payroll_ID)
VALUES (TO_DATE('2025-02-20', 'YYYY-MM-DD'), 'Sick leave', 5, 5);
```

```
-- 5 qualification records
```

```
INSERT INTO QUALIFICATION (position, date_in, emp_ID)
VALUES ('HR Manager', TO_DATE('2018-03-15', 'YYYY-MM-DD'), 1);
```

```
INSERT INTO QUALIFICATION (position, date_in, emp_ID)
VALUES ('Senior Developer', TO_DATE('2019-06-01', 'YYYY-MM-DD'), 2);
```

```
INSERT INTO QUALIFICATION (position, date_in, emp_ID)
VALUES ('Financial Analyst', TO_DATE('2017-09-20', 'YYYY-MM-DD'), 3);
```

```
INSERT INTO QUALIFICATION (position, date_in, emp_ID)
VALUES ('Operations Supervisor', TO_DATE('2020-01-10', 'YYYY-MM-DD'), 4);
```

```
INSERT INTO QUALIFICATION (position, date_in, emp_ID)
VALUES ('Marketing Specialist', TO_DATE('2021-05-25', 'YYYY-MM-DD'), 5);
```

```
-- 10 EMPLOYEE_ADDRESS records
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (1, 'Muscat, Al Khuwair, Street 12, Villa 5');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (2, 'Muscat, Madinat Al Sultan Qaboos, Street 7, Flat 3');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (3, 'Muscat, Ruwi, Street 4, Building 9');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (4, 'Muscat, Qurum, Street 18, Villa 2');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (5, 'Muscat, Al Azaiba, Street 22, Flat 7');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (6, 'Muscat, Bowsher, Street 3, Villa 11');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (7, 'Muscat, Al Ghubrah, Street 9, Flat 1');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
```

```
VALUES (8, 'Muscat, Wattayah, Street 6, Building 4');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (9, 'Muscat, Al Mabelah, Street 14, Villa 8');
```

```
INSERT INTO EMPLOYEE_ADDRESS (emp_ID, contact_add)
VALUES (10, 'Muscat, Al Hail, Street 5, Flat 2');
```

```
-- 5 QUALIFICATION_REQUIREMENTS records
INSERT INTO QUALIFICATION_REQUIREMENTS (qual_ID, requirements)
VALUES (1, 'Bachelor degree in Human Resources or Business Administration');
```

```
INSERT INTO QUALIFICATION_REQUIREMENTS (qual_ID, requirements)
VALUES (2, 'Bachelor degree in Computer Science or Software Engineering');
```

```
INSERT INTO QUALIFICATION_REQUIREMENTS (qual_ID, requirements)
VALUES (3, 'Bachelor degree in Finance or Accounting');
```

```
INSERT INTO QUALIFICATION_REQUIREMENTS (qual_ID, requirements)
VALUES (4, 'Bachelor degree in Business Management or Operations');
```

```
INSERT INTO QUALIFICATION_REQUIREMENTS (qual_ID, requirements)
VALUES (5, 'Bachelor degree in Marketing or Communications');
```

```
-- Filling JOB_DEPARTMENT payroll_ID column
UPDATE JOB_DEPARTMENT SET payroll_ID = 1 WHERE job_ID = 1;
UPDATE JOB_DEPARTMENT SET payroll_ID = 2 WHERE job_ID = 2;
UPDATE JOB_DEPARTMENT SET payroll_ID = 3 WHERE job_ID = 3;
UPDATE JOB_DEPARTMENT SET payroll_ID = 4 WHERE job_ID = 4;
UPDATE JOB_DEPARTMENT SET payroll_ID = 5 WHERE job_ID = 5;
```

```
COMMIT;
```

```
-- =====
-- Task 2
-- =====
```

```
-- Employees between age 25 and 40 ordered by last name
SELECT * FROM EMPLOYEE WHERE AGE>25 AND AGE <40 ORDER BY LNAME ASC;
```

```
-- Employees with payroll total amount greater than 5000
SELECT E.FNAME,E.LNAME,J.NAME, P.* FROM EMPLOYEE E
JOIN JOB_DEPARTMENT J ON E.JOB_ID = J.JOB_ID
```

```
JOIN PAYROLL P ON P.PAYROLL_ID = J.PAYROLL_ID
WHERE TOTAL_AMOUNT>5000;
```

```
-- Employees who took sick leave
SELECT E.FNAME, E.LNAME , L.REASON FROM EMPLOYEE E
JOIN LEAVE L ON E.EMP_ID=L.EMP_ID WHERE LOWER(L.REASON) LIKE
LOWER('%sick%');
```

```
-- Departments and their employees
SELECT D.NAME FROM EMPLOYEE E
JOIN JOB_DEPARTMENT D ON D.JOB_ID = E.JOB_ID WHERE(SELECT JOB_ID FROM
EMPLOYEE);
```

```
-- =====
-- Task 3
-- =====
```

```
-- Give a 10% salary increase to all employees in the 'Engineering' department
UPDATE SALARY_BONUS SET BONUS = BONUS * 1.10
WHERE JOB_ID IN (SELECT JOB_ID FROM JOB_DEPARTMENT WHERE NAME =
'Engineering');
```

```
-- Update the emp_email of all employees to lowercase using Oracle's LOWER() function.
UPDATE EMPLOYEE SET EMP_EMAIL = LOWER(EMP_EMAIL);
```

```
-- Set the salary_range in JOB_DEPARTMENT to 'REVISED' for any department whose
average total payroll
-- exceeds 8000.
UPDATE JOB_DEPARTMENT SET SALARY_RANGE = 'REVISED' WHERE JOB_ID IN
(SELECT D.JOB_ID FROM JOB_DEPARTMENT D
JOIN EMPLOYEE E ON E.JOB_ID = D.JOB_ID
JOIN PAYROLL P ON P.EMP_ID = E.EMP_ID
GROUP BY D.JOB_ID HAVING AVG(P.TOTAL_AMOUNT) > 8000);
```

```
-- =====
-- Task 4
-- =====
```

```
SELECT * FROM LEAVE;
```

```
-- Delete all LEAVE records older than 2 years from today.
DELETE FROM LEAVE WHERE LEAVE_DATE < ADD_MONTHS(SYSDATE, -24);
```

-- Delete QUALIFICATION records for employees who no longer exist in the EMPLOYEE table
DELETE FROM QUALIFICATION WHERE EMP_ID NOT IN
(SELECT EMP_ID FROM EMPLOYEE);

-- Before executing (b), write a SELECT to preview which records would be deleted.
-- After each DELETE, verify the result with a SELECT COUNT(*).
SELECT COUNT(*) FROM QUALIFICATION WHERE EMP_ID NOT IN
(SELECT EMP_ID FROM EMPLOYEE);

04-Aggregation Functions

```
-- =====  
-- 04-Aggregation Functions  
-- =====
```

```
-- =====  
-- Task 1  
-- =====
```

```
-- 1. Total number of employees  
SELECT COUNT(EMP_ID) AS TOTAL_EMPLOYEES FROM EMPLOYEE;
```

```
-- 2. Minimum, maximum and average salary amount  
SELECT MIN(AMOUNT) AS MIN_SALARY FROM SALARY_BONUS;  
SELECT MAX(AMOUNT) AS MAX_SALARY FROM SALARY_BONUS;  
SELECT AVG(AMOUNT) AS AVG_SALARY FROM SALARY_BONUS;
```

```
-- 3. Total bonus amount  
SELECT SUM(BONUS) AS TOTAL_BONUS FROM SALARY_BONUS;
```

```
-- =====  
-- Task 2  
-- =====
```

```
-- 1. Departments where average employee age exceeds 30  
SELECT D.NAME, AVG(E.AGE) AS AVERAGE_AGE FROM JOB_DEPARTMENT D  
JOIN EMPLOYEE E ON E.JOB_ID = D.JOB_ID  
GROUP BY D.JOB_ID, D.NAME  
HAVING AVG(E.AGE) > 30;
```

```
-- 2. Job titles shared by more than 2 employees  
SELECT Q.POSITION, COUNT(Q.EMP_ID) AS EMPLOYEE_COUNT FROM QUALIFICATION  
Q  
GROUP BY Q.POSITION  
HAVING COUNT(Q.EMP_ID) > 2;
```

```
-- 3. Months where total payroll amount exceeds 20000  
SELECT TO_CHAR(PAY_DATE, 'MONTH') AS MONTH_YEAR,  
SUM(TOTAL_AMOUNT) AS TOTAL_PAYROLL FROM PAYROLL  
GROUP BY TO_CHAR(PAY_DATE, 'MONTH')  
HAVING SUM(TOTAL_AMOUNT) > 20000;
```

```
-- =====
```

```
-- Task 3
```

```
-- =====
```

```
SELECT D.NAME, COUNT(E.EMP_ID), SUM(P.TOTAL_AMOUNT), AVG (S.AMOUNT),  
MAX(S.AMOUNT),  
MIN(S.AMOUNT) FROM EMPLOYEE E  
JOIN JOB_DEPARTMENT D ON E.JOB_ID = D.JOB_ID  
JOIN PAYROLL P ON P.PAYROLL_ID = D.PAYROLL_ID  
JOIN SALARY_BONUS S ON S.JOB_ID = E.JOB_ID  
GROUP BY D.NAME;
```

```
-- =====
```

```
-- Task 4
```

```
-- =====
```

```
--a)
```

```
SELECT D.NAME, SUM(P.TOTAL_AMOUNT), AVG(S.AMOUNT) FROM JOB_DEPARTMENT  
D  
JOIN PAYROLL P ON P.PAYROLL_ID = D.PAYROLL_ID  
JOIN SALARY_BONUS S ON P.SALARY_ID = S.SALARY_ID GROUP BY D.NAME  
HAVING SUM(P.TOTAL_AMOUNT) > 15000 AND AVG(S.AMOUNT) > 3000;
```

```
-- b)
```

```
SELECT Q.POSITION, COUNT(E.EMP_ID), AVG(E.AGE) FROM QUALIFICATION Q  
JOIN EMPLOYEE E ON E.EMP_ID =Q.EMP_ID GROUP BY Q.POSITION  
HAVING COUNT(E.EMP_ID)>2 AND AVG(E.AGE)>28;
```

```
--c)
```

```
SELECT E.FNAME || ' ' || E.LNAME AS FULL_NAME, D.NAME AS DEPARTMENT,  
COUNT(L.LEAVE_ID)  
FROM EMPLOYEE E  
JOIN JOB_DEPARTMENT D ON E.JOB_ID = D.JOB_ID  
JOIN LEAVE L ON L.EMP_ID = E.EMP_ID  
GROUP BY E.EMP_ID, E.FNAME || ' ' || E.LNAME, D.NAME  
HAVING COUNT(L.LEAVE_ID) > 1;
```


05-Joins

```
-- =====
-- 05-Joins
-- =====

-- =====
-- Task 1
-- =====

-- Complete employee profile with latest leave date
SELECT E.EMP_ID, E.FNAME || ' ' || E.LNAME AS FULL_NAME, D.NAME AS
DEPARTMENT_NAME,
Q.POSITION AS JOB_TITLE, S.AMOUNT AS SALARY_AMOUNT,
MAX(L.LEAVE_DATE) AS LATEST_LEAVE_DATE FROM EMPLOYEE E
INNER JOIN JOB_DEPARTMENT D ON E.JOB_ID = D.JOB_ID
INNER JOIN SALARY_BONUS S ON E.SALARY_ID = S.SALARY_ID
INNER JOIN QUALIFICATION Q ON E.EMP_ID = Q.EMP_ID
INNER JOIN PAYROLL P ON E.EMP_ID = P.EMP_ID
INNER JOIN LEAVE L ON E.EMP_ID = L.EMP_ID
GROUP BY E.EMP_ID, E.FNAME || ' ' || E.LNAME, D.NAME, Q.POSITION, S.AMOUNT;

-- =====
-- Task 2
-- =====

-- 1. Employees who have never taken any leave
SELECT E.EMP_ID, E.FNAME, E.LNAME FROM EMPLOYEE E
LEFT JOIN LEAVE L ON L.EMP_ID = E.EMP_ID
WHERE L.LEAVE_ID IS NULL;

-- 2. Departments that have no salary/bonus records
SELECT J.JOB_ID, J.NAME FROM JOB_DEPARTMENT J
LEFT JOIN SALARY_BONUS S ON S.JOB_ID = J.JOB_ID
WHERE S.JOB_ID IS NULL;

-- =====
-- Task 3
-- =====

SELECT P.PAYROLL_ID, E.FNAME || ' ' || E.LNAME AS FULL_NAME, D.NAME, Q.POSITION,
S.AMOUNT, S.BONUS, L.REASON, P.TOTAL_AMOUNT FROM PAYROLL P
```

```
JOIN EMPLOYEE E ON P.EMP_ID = E.EMP_ID
JOIN JOB_DEPARTMENT D ON E.JOB_ID = D.JOB_ID
JOIN SALARY_BONUS S ON P.SALARY_ID = S.SALARY_ID
LEFT JOIN QUALIFICATION Q ON Q.EMP_ID = E.EMP_ID
LEFT JOIN LEAVE L ON L.PAYROLL_ID = P.PAYROLL_ID
ORDER BY D.NAME, P.TOTAL_AMOUNT DESC;
```

```
-- =====
```

```
-- Task 4
```

```
-- =====
```

```
-- adding column
```

```
ALTER TABLE EMPLOYEE ADD (manager_id NUMBER REFERENCES
EMPLOYEE(emp_ID));
```

```
-- a.
```

```
UPDATE EMPLOYEE SET manager_id = 1 WHERE emp_ID = 2;
UPDATE EMPLOYEE SET manager_id = 1 WHERE emp_ID = 6;
UPDATE EMPLOYEE SET manager_id = 2 WHERE emp_ID = 7;
UPDATE EMPLOYEE SET manager_id = 3 WHERE emp_ID = 8;
UPDATE EMPLOYEE SET manager_id = 4 WHERE emp_ID = 9;
UPDATE EMPLOYEE SET manager_id = 5 WHERE emp_ID = 10;
```

```
-- b.
```

```
SELECT E.EMP_ID, E.FNAME || ' ' || E.LNAME AS EMPLOYEE_NAME,
M.FNAME || ' ' || M.LNAME AS MANAGER_NAME FROM EMPLOYEE E
LEFT JOIN EMPLOYEE M ON E.MANAGER_ID = M.EMP_ID
ORDER BY E.EMP_ID;
```

```
-- c.
```

```
SELECT DISTINCT M.EMP_ID, M.FNAME, M.LNAME FROM EMPLOYEE M
JOIN EMPLOYEE E ON E.MANAGER_ID = M.EMP_ID;
```

06-Subqueries

```
-- =====
-- 06-Subqueries
-- =====

-- =====
-- Task 1
-- =====

-- 1. Employees earning above company average salary
SELECT E.EMP_ID, E.FNAME || ' ' || E.LNAME AS FULL_NAME,
D.NAME, S.AMOUNT AS SALARY FROM EMPLOYEE E
JOIN SALARY_BONUS S ON E.SALARY_ID = S.SALARY_ID
JOIN JOB_DEPARTMENT D ON E.JOB_ID = D.JOB_ID
WHERE S.AMOUNT > (SELECT AVG(SB.AMOUNT) FROM SALARY_BONUS SB);

-- 2. Department with the highest total payroll amount
SELECT D.NAME, SUM(P.TOTAL_AMOUNT)
FROM JOB_DEPARTMENT D
JOIN PAYROLL P ON P.PAYROLL_ID = D.PAYROLL_ID
GROUP BY D.NAME
HAVING SUM(P.TOTAL_AMOUNT) = (SELECT MAX(TOTAL_AMOUNT) FROM PAYROLL);

-- =====
-- Task 2
-- =====

-- 1. Employees in departments with bonus greater than 500
SELECT E.EMP_ID, E.FNAME, E.LNAME FROM EMPLOYEE E
WHERE E.JOB_ID IN (SELECT S.JOB_ID FROM SALARY_BONUS S
WHERE S.BONUS > 500);

-- 2. Employees earning more than ALL salaries in Maintenance department
SELECT E.EMP_ID, E.FNAME, E.LNAME, S.AMOUNT FROM EMPLOYEE E
JOIN SALARY_BONUS S ON E.SALARY_ID = S.SALARY_ID
WHERE S.AMOUNT > ALL (SELECT S2.AMOUNT FROM SALARY_BONUS S2
JOIN JOB_DEPARTMENT D ON D.JOB_ID = S2.JOB_ID
WHERE D.NAME = 'Maintenance');

-- 3. Employees earning more than ANY salary in HR department
SELECT E.EMP_ID, E.FNAME, E.LNAME, S.AMOUNT FROM EMPLOYEE E
JOIN SALARY_BONUS S ON E.SALARY_ID = S.SALARY_ID
```

```
WHERE S.AMOUNT > ANY (SELECT S2.AMOUNT FROM SALARY_BONUS S2
JOIN JOB_DEPARTMENT D ON D.JOB_ID = S2.JOB_ID
WHERE D.NAME = 'Human Resources');
```

```
-- =====
-- Task 3
-- =====
```

```
--a.
SELECT E.EMP_ID, E.FNAME || ' ' || E.LNAME AS FULL_NAME,
(SELECT COUNT(*) FROM PAYROLL P WHERE P.EMP_ID = E.EMP_ID) AS
PAYROLL_COUNT
FROM EMPLOYEE E;
```

```
--b.
SELECT E.EMP_ID, E.FNAME || ' ' || E.LNAME AS FULL_NAME,
(SELECT COUNT(*) FROM LEAVE L WHERE L.EMP_ID = E.EMP_ID) AS LEAVE_COUNT
FROM EMPLOYEE E WHERE (SELECT COUNT(*) FROM LEAVE L WHERE L.EMP_ID =
E.EMP_ID)
> (SELECT COUNT(*) FROM LEAVE) / (SELECT COUNT(*) FROM EMPLOYEE);
```

```
-- =====
-- Task 4
-- =====
```

```
-- a.
SELECT D.NAME
FROM JOB_DEPARTMENT D
WHERE EXISTS (SELECT 1 FROM EMPLOYEE E
JOIN PAYROLL P ON P.EMP_ID = E.EMP_ID
WHERE E.JOB_ID = D.JOB_ID AND P.TOTAL_AMOUNT > 10000
);
```

```
--b.
SELECT E.EMP_ID, E.FNAME, E.LNAME
FROM EMPLOYEE E
WHERE NOT EXISTS ( SELECT * FROM QUALIFICATION Q WHERE Q.EMP_ID =
E.EMP_ID);
```

07 Views

```
-- =====
-- 07 Views
-- =====

-- =====
-- Task 1
-- =====

-- View creation
CREATE VIEW VW_EMPLOYEE_SUMMARY AS
SELECT E.EMP_ID,E.FNAME || ' ' || E.LNAME AS
FULL_NAME,E.GENDER,E.AGE,D.NAME,Q.POSITION FROM EMPLOYEE E
JOIN JOB_DEPARTMENT D ON E.JOB_ID = D.JOB_ID
JOIN QUALIFICATION Q ON Q.EMP_ID=E.EMP_ID;

-- (a) Query the view to list all female employees over 30
SELECT * FROM VW_EMPLOYEE_SUMMARY WHERE GENDER ='F' AND AGE>30;

-- (b) Attempt to INSERT a row through the view
INSERT INTO VW_EMPLOYEE_SUMMARY (FULL_NAME, GENDER, AGE, NAME,
POSITION)
VALUES ('Tibyan Saad', 'F', 24, 'IT', 'Trainee');

-- Error starting at line : 8 in command -
-- INSERT INTO VW_EMPLOYEE_SUMMARY (FULL_NAME, GENDER, AGE, NAME,
POSITION) VALUES ('Tibyan Saad', 'F', 24, 'IT', 'Trainee')
-- Error at Command Line : 8 Column : 34
-- Error report -
-- SQL Error: ORA-01733: virtual column not allowed here
-- https://docs.oracle.com/error-help/db/ora-01733/01733.00000 - "virtual column not allowed
here"
-- *Cause: An attempt was made to use an INSERT, UPDATE, or DELETE statement on an
expression in a view.
-- *Action: INSERT, UPDATE, or DELETE data in the base tables, instead of the view.
-- More Details :
-- https://docs.oracle.com/error-help/db/ora-01733/

-- =====
-- Task 2
-- =====
```

-- View creation

```
CREATE VIEW VW_PAYROLL_DASHBOARD AS
SELECT P.PAYROLL_ID, E.FNAME || ' ' || E.LNAME AS FULL_NAME,
D.NAME,S.AMOUNT,S.BONUS,
L.REASON,P.PAY_DATE,P.TOTAL_AMOUNT FROM EMPLOYEE E
JOIN JOB_DEPARTMENT D ON D.JOB_ID = E.JOB_ID
JOIN PAYROLL P ON P.PAYROLL_ID =D.PAYROLL_ID
JOIN SALARY_BONUS S ON S.JOB_ID = D.JOB_ID
JOIN LEAVE L ON L.PAYROLL_ID = P.PAYROLL_ID;
```

-- Query the view to find the top 5 payroll records by total_amount.

```
SELECT * FROM VW_PAYROLL_DASHBOARD ORDER BY TOTAL_AMOUNT DESC FETCH
FIRST 5 ROWS ONLY;
```

08 Stored Procedures & Functions

```
-- =====
-- 08 Stored Procedures & Functions
-- =====

-- =====
-- Task 1
-- =====

-- Create procedure
CREATE OR REPLACE PROCEDURE SP_ADD_EMPLOYEE (
    P_FNAME    IN EMPLOYEE.FNAME%TYPE,
    P_LNAME    IN EMPLOYEE.LNAME%TYPE,
    P_GENDER   IN EMPLOYEE.GENDER%TYPE,
    P_AGE      IN EMPLOYEE.AGE%TYPE,
    P_EMP_EMAIL IN EMPLOYEE.EMP_EMAIL%TYPE,
    P_EMP_PASS  IN EMPLOYEE.EMP_PASS%TYPE,
    P_JOB_ID   IN EMPLOYEE.JOB_ID%TYPE,
    P_SALARY_ID IN EMPLOYEE.SALARY_ID%TYPE
)
IS
BEGIN
    INSERT INTO EMPLOYEE (FNAME, LNAME, GENDER, AGE, EMP_EMAIL, EMP_PASS,
JOB_ID, SALARY_ID)
    VALUES (P_FNAME, P_LNAME, P_GENDER, P_AGE, P_EMP_EMAIL, P_EMP_PASS,
P_JOB_ID, P_SALARY_ID);

    DBMS_OUTPUT.PUT_LINE('Employee ' || P_FNAME || ' ' || P_LNAME || ' added
successfully.');
```

COMMIT;

```
EXCEPTION
    -- I already have a unique constraint for email during table creation
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: Email already exists in the system.');
```

END;

/

-- Testing

```
SET SERVEROUTPUT ON;
EXEC SP_ADD_EMPLOYEE('Salim', 'Al-Kindi', 'M',27,
'salim.kindi@ems.com','Pass@5678',1,1);
EXEC SP_ADD_EMPLOYEE('Salim', 'Al-Kindi', 'M',27,
'salim.kindi@ems.com','Pass@5678',1,1);
```

```
-- ERROR at line 1:
-- ORA-20001: Error: Email already exists in the system.
-- ORA-06512: at "PROJ.SP_ADD_EMPLOYEE", line 21
-- ORA-06512: at line 1
```

```
-- =====
-- Task 2
-- =====
```

```
-- Create procedure
CREATE OR REPLACE FUNCTION FN_NET_SALARY (
  P_EMP_ID IN EMPLOYEE.EMP_ID%TYPE
)
RETURN NUMBER
IS
  V_NET_SALARY NUMBER;
BEGIN
  SELECT S.AMOUNT + S.BONUS INTO V_NET_SALARY
  FROM EMPLOYEE E
  JOIN SALARY_BONUS S ON E.SALARY_ID = S.SALARY_ID
  WHERE E.EMP_ID = P_EMP_ID;

  RETURN V_NET_SALARY;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END;
/
```

```
-- using procedure in a select statement
SELECT E.EMP_ID,E.FNAME || ' ' || E.LNAME AS FULL_NAME,FN_NET_SALARY(E.EMP_ID)
AS NET_SALARY
FROM EMPLOYEE E ORDER BY NET_SALARY DESC;
```


09 Triggers

```
-- =====
-- 09 Triggers
-- =====

-- =====
-- Task 1
-- =====

-- removing the default sequence added at table creation
ALTER TABLE EMPLOYEE MODIFY (EMP_ID NUMBER);

-- creating sequence
CREATE SEQUENCE EMP_SEQ START WITH 11 INCREMENT BY 1;

-- creating trigger
CREATE OR REPLACE TRIGGER TRG_EMP_ID
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
WHEN (NEW.EMP_ID IS NULL)
BEGIN
    SELECT EMP_SEQ.NEXTVAL INTO :NEW.EMP_ID FROM DUAL;
END;
/

-- tesing the trigger
INSERT INTO EMPLOYEE (FNAME, LNAME, GENDER, AGE, EMP_EMAIL, EMP_PASS,
JOB_ID, SALARY_ID)
VALUES ('Lina', 'Al-Busaidi', 'F', 27, 'lina.busaidi@ems.com', 'Pass@9012', 2, 2);

INSERT INTO EMPLOYEE (FNAME, LNAME, GENDER, AGE, EMP_EMAIL, EMP_PASS,
JOB_ID, SALARY_ID)
VALUES ('Mariam', 'Al-Riyami', 'F', 26, 'mariam.riyami@ems.com', 'Pass@3344', 3, 3);

-- checking
SELECT * FROM EMPLOYEE;
```

	EMP_ID	FNAME	LNAME	GENDER	AGE	EMP_EMAIL	EMP_PASS	JOB_ID	SALARY_ID
1	1	Mohammed	Al-Rashidi	M	35	mohammed.rashidi@ems.com	Pass@1234	1	1
2	2	Fatima	Al-Balushi	F	28	fatima.balushi@ems.com	Pass@1234	2	2
3	3	Ahmed	Al-Harathi	M	42	ahmed.harathi@ems.com	Pass@1234	3	3
4	4	Maryam	Al-Zadjali	F	31	maryam.zadjali@ems.com	Pass@1234	4	4
5	5	Khalid	Al-Farsi	M	38	khalid.farsi@ems.com	Pass@1234	5	5
6	6	Noura	Al-Habsi	F	26	noura.habsi@ems.com	Pass@1234	1	1
7	7	Omar	Al-Maskari	M	33	omar.maskari@ems.com	Pass@1234	2	2
8	8	Aisha	Al-Lawati	F	29	aisha.lawati@ems.com	Pass@1234	3	3
9	9	Yousuf	Al-Mamari	M	45	yousuf.mamari@ems.com	Pass@1234	4	4
10	10	Hessa	Al-Siyabi	F	30	hessa.siyabi@ems.com	Pass@1234	5	5
11	11	Salim	Al-Kindi	M	27	salim.kind@ems.com	Pass@5678	1	1
12	13	Mariam	Al-Riyami	F	26	mariam.riyami@ems.com	Pass@3344	3	3
13	12	Lina	Al-Busaidi	F	27	lina.busaidi@ems.com	Pass@9012	2	2

```
-- =====
```

```
-- Task 2
```

```
-- =====
```

```
-- creating EMPLOYEE_LOG table w/sequence
```

```
CREATE SEQUENCE employee_log_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE EMPLOYEE_LOG (
log_id NUMBER DEFAULT employee_log_seq.NEXTVAL PRIMARY KEY,
emp_id NUMBER NOT NULL,
action VARCHAR2(50) NOT NULL,
log_timestamp TIMESTAMP DEFAULT SYSTIMESTAMP,
CONSTRAINT fk_log_emp FOREIGN KEY (emp_id)
REFERENCES EMPLOYEE(emp_id)
);
```

```
-- creating trigger
```

```
CREATE OR REPLACE TRIGGER TRG_EMP_WELCOME_LOG
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO EMPLOYEE_LOG (emp_id, action)
    VALUES (:NEW.EMP_ID, 'NEW HIRE');
END;
/
```

-- inserting records for testing

```
INSERT INTO EMPLOYEE (FNAME, LNAME, GENDER, AGE, EMP_EMAIL, EMP_PASS,  
JOB_ID, SALARY_ID)
```

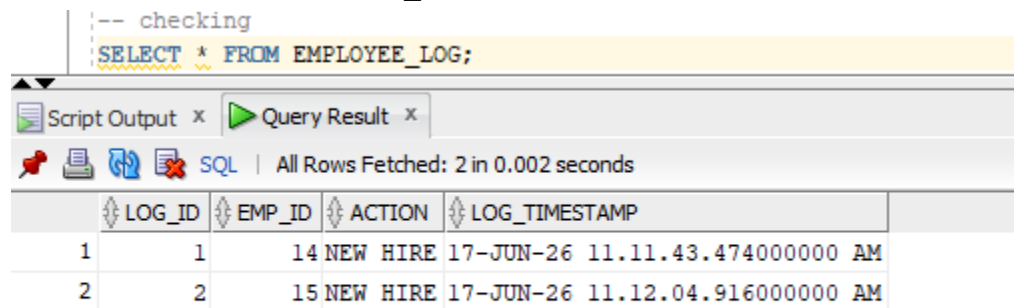
```
VALUES ('Yaqoub', 'Al-Amri', 'M', 32, 'yaqoub.amri@ems.com', 'Pass@7788', 4, 4);
```

```
INSERT INTO EMPLOYEE (FNAME, LNAME, GENDER, AGE, EMP_EMAIL, EMP_PASS,  
JOB_ID, SALARY_ID)
```

```
VALUES ('Salma', 'Al-Wahaibi', 'F', 29, 'salma.wahaibi@ems.com', 'Pass@4455', 5, 5);
```

-- checking

```
SELECT * FROM EMPLOYEE_LOG;
```



The screenshot shows a SQL query execution window. The query entered is `SELECT * FROM EMPLOYEE_LOG;`. The results are displayed in a table with 4 columns: LOG_ID, EMP_ID, ACTION, and LOG_TIMESTAMP. There are 2 rows of data.

LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
1	1	14 NEW HIRE	17-JUN-26 11.11.43.474000000 AM
2	2	15 NEW HIRE	17-JUN-26 11.12.04.916000000 AM

10 Oracle Scheduler Jobs

```
-- =====
-- 10 Oracle Scheduler Jobs
-- =====

-- =====
-- Task 1
-- =====

-- creating the job
BEGIN
  DBMS_SCHEDULER.CREATE_JOB(
    job_name      => 'JOB_GREET_EMPLOYEES',
    job_type      => 'PLSQL_BLOCK',
    job_action     => 'BEGIN
                        DBMS_OUTPUT.PUT_LINE("Payroll System Initialized");
                        INSERT INTO EMPLOYEE_LOG (emp_id, action)
                        VALUES (1, "SYSTEM JOB EXECUTED");
                        END;',
    start_date     => SYSTIMESTAMP + INTERVAL '2' MINUTE, --once in 2 mins
    enabled        => TRUE
  );
END;
/

-- testing after 2 mins
SELECT * FROM EMPLOYEE_LOG;
```



The screenshot shows the Oracle SQL Developer interface. The 'Query Results' window is active, displaying the results of the SQL query 'SELECT * FROM EMPLOYEE_LOG;'. The window title is 'Query Result x'. Below the title bar, there are icons for saving, refreshing, and other query actions, followed by the text 'SQL | All Rows Fetched: 3 in 0.002 seconds'. The results are shown in a table with 5 columns: LOG_ID, EMP_ID, ACTION, and LOG_TIMESTAMP. The first three columns are highlighted with a double-headed arrow icon. The table contains three rows of data.

LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
1	1	14 NEW HIRE	17-JUN-26 11.11.43.474000000 AM
2	2	15 NEW HIRE	17-JUN-26 11.12.04.916000000 AM
3	3	1 SYSTEM JOB EXECUTED	17-JUN-26 11.23.52.234000000 AM

```
-- =====
-- Task 2
-- =====
```

```
-- creating job
```

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB(
    job_name      => 'JOB_DAILY_LEAVE_REPORT',
    job_type      => 'PLSQL_BLOCK',
    job_action    => 'DECLARE
                        V_LEAVE_COUNT NUMBER;
                      BEGIN
                        SELECT COUNT(*) INTO V_LEAVE_COUNT
                        FROM LEAVE
                        WHERE TRUNC(LEAVE_DATE) = TRUNC(SYSDATE);
                        INSERT INTO EMPLOYEE_LOG (emp_id, action)
                        VALUES (1, "DAILY LEAVE REPORT: " || V_LEAVE_COUNT || " leave
record(s) today");
                      END;';
    start_date    => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0',
    enabled       => TRUE
  );
END;
/
```

```
-- showing job definition
```

```
SELECT * FROM USER_SCHEDULER_JOBS WHERE JOB_NAME =
'JOB_DAILY_LEAVE_REPORT';
```

-- showing job definition

```
SELECT * FROM USER_SCHEDULER_JOBS WHERE JOB_NAME =
'JOB_DAILY_LEAVE_REPORT';
```

Script Output: * Query Result: *

All Rows Fetched: 1 in 0.12 seconds

JOB_NAME	JOB_SUBNAME	JOB_STYLE	JOB_CREATOR	CLIENT_ID	GLOBAL_UID	PROGRAM_OWNER	PROGRAM_NAME	JOB_TYPE	JOB_ACTION
JOB_DAILY_LEAVE_REPORT	(null)	REGULAR	PROD	(null)	(null)	(null)	(null)	PLSQL_BLOCK DECLARE	V_LEAVE_COUNT NUMBER; BEGIN